

Guião 08 – Balanceamento de Carga e Escalabilidade no Kubernetes

Infraestruturas de Computação na Nuvem, Aula 08

1 Objetivos

- Implantar uma aplicação que identifica qual a instância que respondeu, para observar a distribuição de carga.
- Verificar que o Service do Kubernetes distribui pedidos pelas réplicas disponíveis.
- Configurar *liveness* e *readiness probes* e observar o efeito de uma instância deixar de estar saudável.
- Escalar o serviço manualmente e, de seguida, automaticamente com um Horizontal Pod Autoscaler (HPA).
- Executar um *rolling update* sem interrupção de serviço e observar o comportamento durante a atualização.

2 Pré-requisitos e Material

- O cluster Kubernetes de dois nós montado no Guião 05, com `kubect1` configurado no control plane.
- Acesso ao Proxmox, para simular a falha de um nó no exercício final.
- Para a parte do HPA: o `metrics-server` instalado no cluster (o guião indica como fazê-lo).

Se o cluster da Aula 05 já não estiver disponível: volte a segui-lo para o remontar, ou peça à docente o cluster de demonstração. Este guião assume-o como ponto de partida.

3 Introdução

Na Aula 05 vimos o Kubernetes como orquestrador; nesta aula usamo-lo para estudar concretamente disponibilidade e escalabilidade. O objeto central é o **Service**: além de dar um ponto de acesso estável, ele funciona como balanceador de carga interno, distribuindo pedidos pelas réplicas saudáveis de uma aplicação.

Vamos implantar uma aplicação que devolve o nome do Pod que respondeu – o que torna a distribuição de carga imediatamente visível –, configurar health checks, e observar o que acontece quando uma réplica adoece, quando a carga aumenta, e quando se atualiza a aplicação em produção.

4 Parte 1 – Implantar uma aplicação identificável

1. **Criar o manifesto `app.yaml`.** Usamos a imagem `hashicorp/http-echo`, que responde com um texto fixo, e a variável `POD_NAME` obtida da própria API do Kubernetes (*downward API*) para identificar a instância:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginxdemos/hello:plain-text
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "50m"
              memory: "32Mi"
            limits:
              cpu: "200m"
              memory: "128Mi"
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 2
            periodSeconds: 3
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: NodePort

```

A imagem `nginxdemos/hello:plain-text` devolve, em texto simples, o nome do servidor que respondeu – exatamente o que precisamos para ver a distribuição.

2. Aplicar e confirmar:

```

$ kubectl apply -f app.yaml
$ kubectl get pods -o wide

```

```
$ kubectl get svc web-svc
```

Registe a porta NodePort atribuída e em que nós os Pods ficaram colocados.

3. **Observar a distribuição de carga.** Faça vários pedidos seguidos e conte quantas vezes cada Pod respondeu:

```
$ for i in $(seq 1 30); do \  
    curl -s http://localhost:<nodeport> | grep -i "server name"; \  
done | sort | uniq -c
```

Deverá ver os pedidos repartidos pelas três réplicas. Registe a contagem obtida.

4. **Confirmar que o balanceamento funciona a partir de qualquer nó:** repita a partir do IP do outro nó do cluster e compare.

5 Parte 2 – Health checks em ação

5. **Observar os *endpoints* do Service.** O Service só encaminha tráfego para Pods considerados prontos:

```
$ kubectl get endpoints web-svc
```

Registe quantos endereços estão listados.

6. **Tornar um Pod não-saudável.** Escolha um Pod e sabote o conteúdo servido, de modo a que a probe deixe de obter resposta:

```
$ kubectl get pods -l app=web  
$ kubectl exec -it <nome-do-pod> -- sh -c "kill 1" || true
```

Em alternativa, mais controlado, edite temporariamente o Deployment para apontar a `readinessProbe` para um caminho inexistente:

```
$ kubectl patch deployment web --type=json \  
-p='[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/  
httpGet/path","value":"/nao-existe"}]'
```

7. **Observar a reação do cluster:**

```
$ kubectl get pods -l app=web  
$ kubectl get endpoints web-svc  
$ kubectl describe pod <nome-do-pod> | tail -20
```

Repare na coluna `READY` (ex. 0/1) e na diminuição do número de endpoints: os Pods que falham a readiness probe são retirados do balanceamento, mas *não* são reiniciados.

8. **Confirmar que o serviço continua disponível** enquanto houver réplicas saudáveis, e depois reverter a alteração:

```
$ kubectl patch deployment web --type=json \  
-p='[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/  
httpGet/path","value":"/"}]'  
$ kubectl get endpoints web-svc
```

9. **Distinguir readiness de liveness.** Consulte a contagem de reinícios dos Pods:

```
$ kubectl get pods -l app=web
```

A coluna `RESTARTS` regista as ações da *liveness* probe (que reinicia o contentor), não da *readiness* (que apenas o retira do tráfego).

6 Parte 3 – Escalar

10. Escalamento manual (horizontal):

```
$ kubectl scale deployment web --replicas=6
$ kubectl get pods -o wide
$ kubectl get endpoints web-svc
```

Repita a contagem de distribuição da Parte 1 e compare com o resultado obtido com 3 réplicas.

11. Instalar o metrics-server, necessário para o auto-scaling:

```
$ kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/
latest/download/components.yaml
```

Num cluster de laboratório com certificados autoassinados, é normalmente necessário permitir TLS inseguro:

```
$ kubectl patch deployment metrics-server -n kube-system --type=json \
-p='[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--
kubectlet-insecure-tls"}]'
```

Aguarde e confirme que as métricas ficam disponíveis:

```
$ kubectl top nodes
$ kubectl top pods
```

12. Criar o Horizontal Pod Autoscaler:

```
$ kubectl scale deployment web --replicas=2
$ kubectl autoscale deployment web --cpu-percent=50 --min=2 --max=8
$ kubectl get hpa
```

13. Gerar carga para provocar o escalamento automático. Num terminal separado, lance um Pod que faça pedidos continuamente:

```
$ kubectl run gerador-carga --rm -it --image=busybox:1.36 --restart=Never -- \
sh -c "while true; do wget -q -O- http://web-svc; done"
```

14. Observar o HPA a reagir, noutro terminal:

```
$ kubectl get hpa --watch
```

Acompanhe a coluna de utilização de CPU e o número de réplicas. Registe quanto tempo demorou até serem criadas novas réplicas.

15. Parar a carga (Ctrl+C no gerador) e observar o *scale-down*. Note que a redução é deliberadamente mais lenta do que o aumento, para evitar oscilações:

```
$ kubectl get hpa --watch
```

7 Parte 4 – Rolling update sem downtime

16. **Preparar a monitorização contínua.** Num terminal, faça pedidos ininterruptos ao serviço para detetar qualquer falha durante a atualização:

```
$ while true; do \  
    curl -s -o /dev/null -w "%{http_code} " http://localhost:<nodeport>; \  
    sleep 0.3; \  
done
```

17. **Desencadear um rolling update** noutro terminal, alterando a imagem:

```
$ kubectl set image deployment/web web=nginxdemos/hello:latest  
$ kubectl rollout status deployment/web
```

18. **Observar a substituição gradual**, num terceiro terminal:

```
$ kubectl get pods -l app=web --watch
```

Repare que os Pods novos só recebem tráfego depois de passarem a readiness probe, e que os antigos só são removidos depois disso. Verifique no primeiro terminal se apareceu algum código diferente de 200.

19. **Consultar o histórico e reverter:**

```
$ kubectl rollout history deployment/web  
$ kubectl rollout undo deployment/web  
$ kubectl rollout status deployment/web
```

20. **Teste final: falha de um nó com carga ativa.** Mantendo o curl contínuo a correr, pare a VM do worker a partir do Proxmox:

```
$ qm stop <vmid-w1>
```

Observe no terminal de monitorização se houve interrupção de serviço, e no cluster a redistribuição:

```
$ kubectl get nodes  
$ kubectl get pods -o wide
```

Volte a arrancar a VM no final:

```
$ qm start <vmid-w1>
```

21. **Limpeza:**

```
$ kubectl delete hpa web  
$ kubectl delete -f app.yaml
```

8 Entregável / Questões de Verificação

- Submeta o `app.yaml` e a contagem de distribuição de pedidos (`sort | uniq -c`) obtida com 3 e com 6 réplicas.
- Qual a diferença de comportamento entre a *readiness probe* e a *liveness probe*? Que evidência concreta observou que distingue as duas?

- c) Quando um Pod deixou de passar a readiness probe, o que aconteceu ao número de *end-points* do Service? Que conceito da aula teórica isto ilustra?
- d) Registe a evolução do HPA (utilização de CPU e número de réplicas) durante o teste de carga. Quanto tempo demorou a reagir? Trata-se de uma política reativa ou preditiva?
- e) Porque é que o *scale-down* é mais lento do que o *scale-up*? Que problema esta assimetria evita?
- f) Durante o rolling update, observou algum código HTTP diferente de 200? Explique, com base nas probes, porque é que o serviço se manteve disponível.
- g) Ao parar a VM do worker com carga ativa, houve interrupção? Compare o comportamento observado com o que aconteceria se a aplicação fosse *stateful* com sessões em memória.
- h) Este serviço é *stateless*. Que alterações seriam necessárias para escalar horizontalmente uma aplicação que guarda sessões de utilizador em memória?
- i) O Service do Kubernetes faz balanceamento de camada 4 ou de camada 7? Justifique, e indique que recurso do Kubernetes usaria se precisasse de encaminhar por caminho de URL.